

Lecture 24: Unsupervised Learning

CMSC/DATA 320: Introduction to Data Science

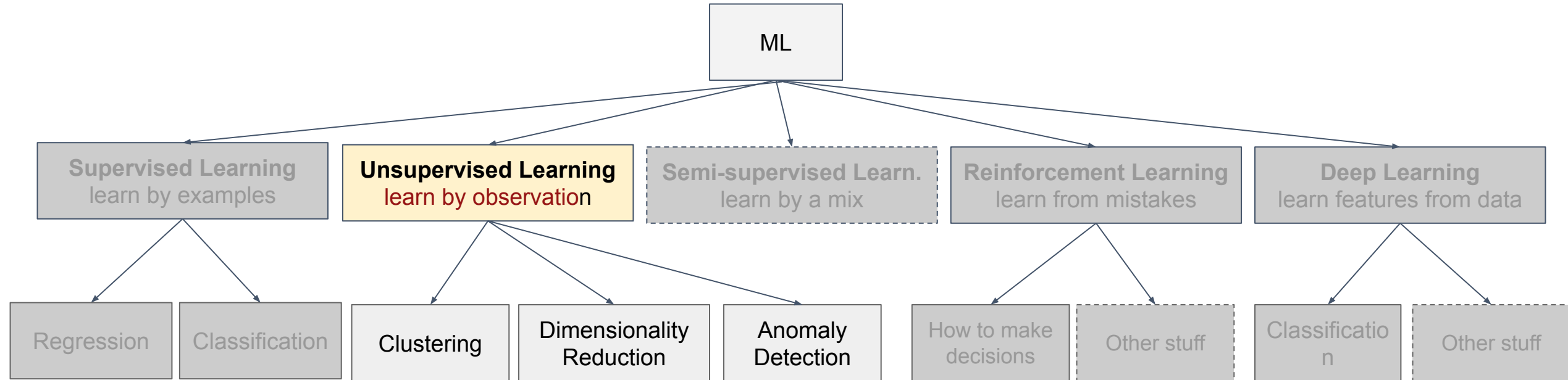
Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA6, WA6 due Sun April 26
 - Q12, WA5, PA5 grades out
 - A note about the final
 - Questions similar in style to the quizzes: MCQ and fill in the blank
 - ~90 min, but we'll have 2 hours
 - You are allowed 1 side of a letter-sized piece of paper of handwritten notes
- 

Unsupervised Learning

Main categories of Machine Learning



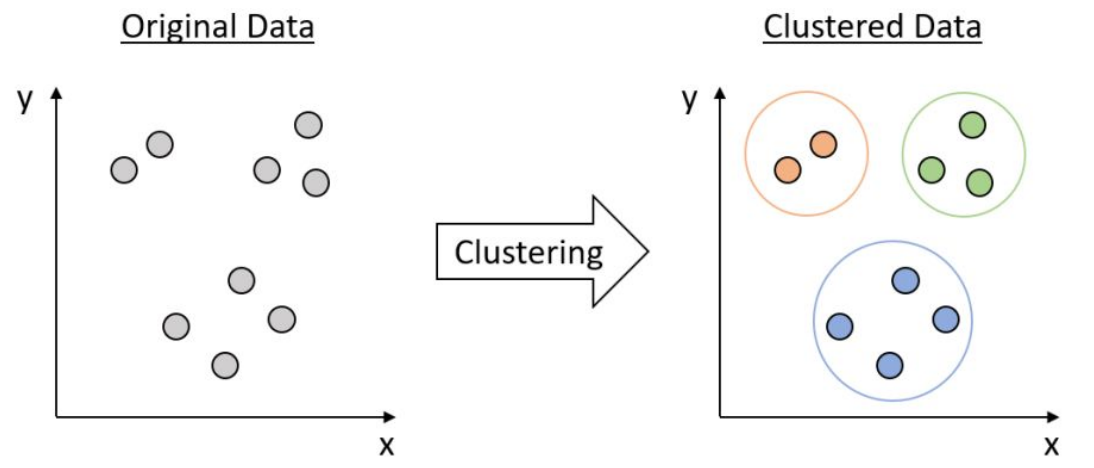
Clustering



Clustering

Also called **cluster analysis**, clustering is a powerful unsupervised learning technique used to identify natural groupings (cluster) within datasets.

- **Group similar data points** or objects into clusters or categories.
- By grouping data points with similar characteristics, cluster analysis helps **reveal hidden patterns, trends, and relationships.**



Similarity in cluster analysis

In clustering, the definition of similarity **depends on the technique you're using**.

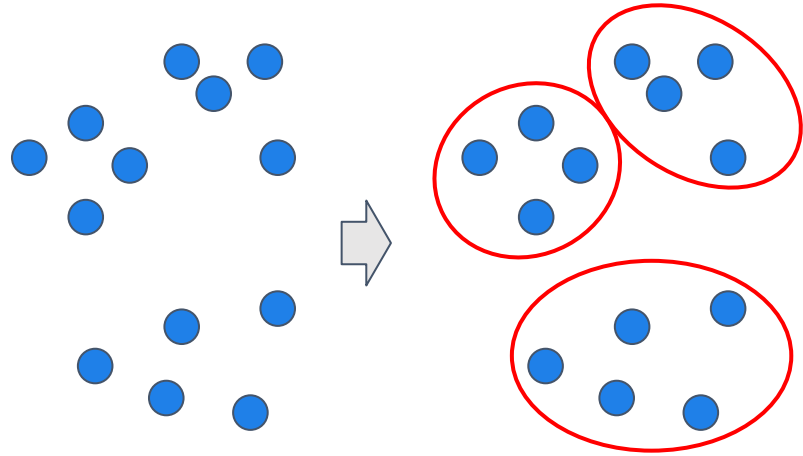
In general, we use measures of **proximity** (distance) specific to the problem at hand. Some examples:

- **Centroid-based clustering**
 - Proximity is defined by **distance from a centroid**
 - The centroid is the cluster **center**, which is the **mean** of all data points in the cluster.
- **Hierarchical clustering** (HCA)
 - Proximity is **distance between data points**
 - Also considers **distances between clusters**
- **Density-based clustering**
 - Proximity is **distance between data points in data space** (a contiguous region of high data point density)
 - Basically how densely packed together data points are

Types of Clustering

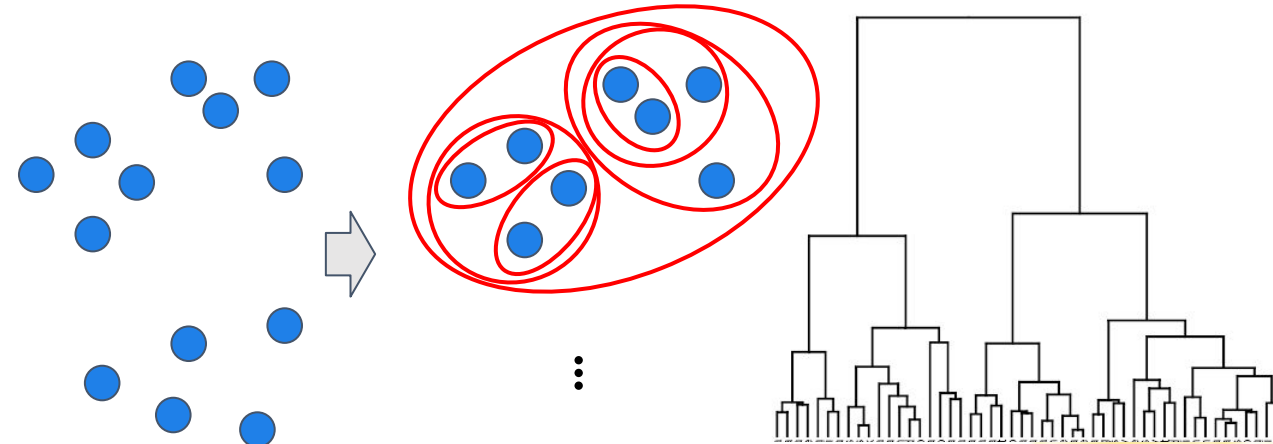
1. **Partitional Clustering**

Data is partitioned into non-overlapping subsets (clusters) such that each data point is in exactly one subset



2. **Hierarchical clustering**

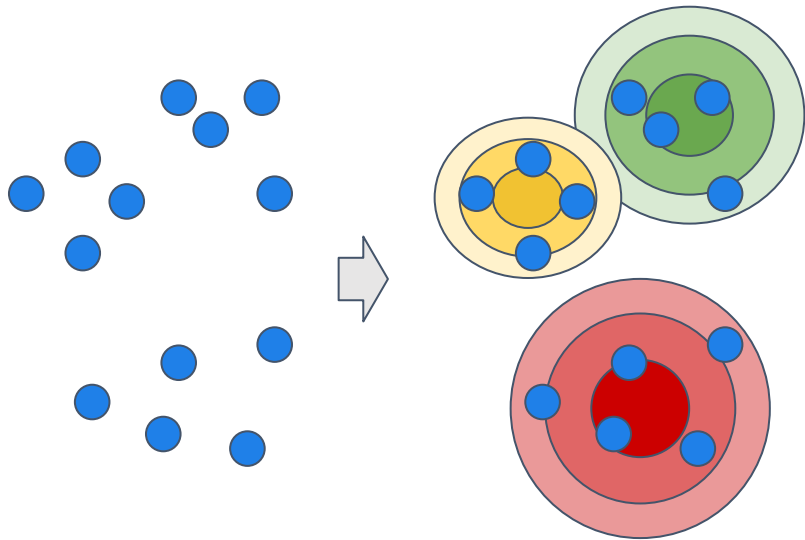
A set of nested clusters is organized hierarchically as a tree



Types of Clustering

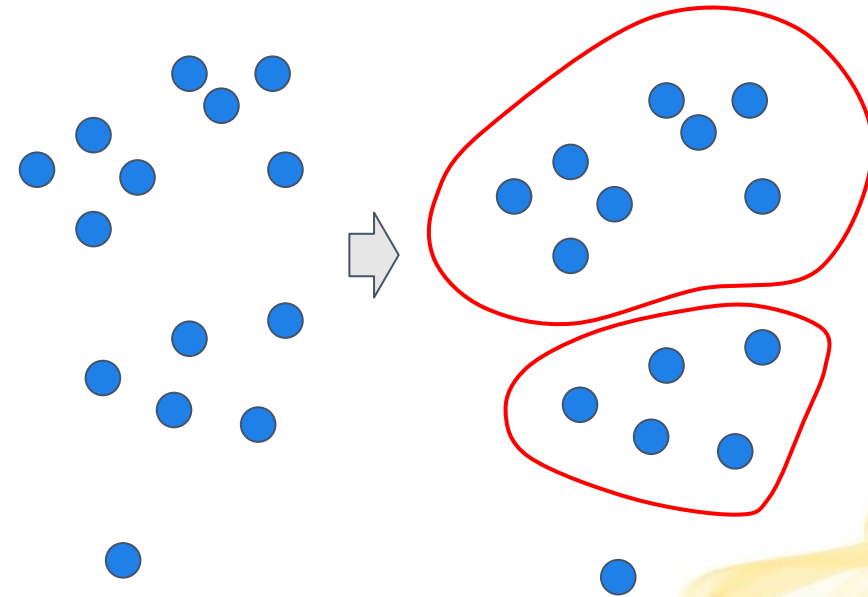
3. **Model-Based Clustering**

Data is clustered into homogeneous groups using probability distributions.



4. **Density clustering**

Data is grouped by areas of high density separated by areas of low density.



Most Common Clustering Algorithms

- ❑ **K-Means** - centroid-based clustering
 - ❑ **Agglomerative** (AGNES) - hierarchical clustering
 - ❑ **Density Based Scan Clustering** (DBSCAN) - density clustering
 - ❑ **Gaussian Mixture Model** - probabilistic model-based clustering
- 

K-Means Clustering

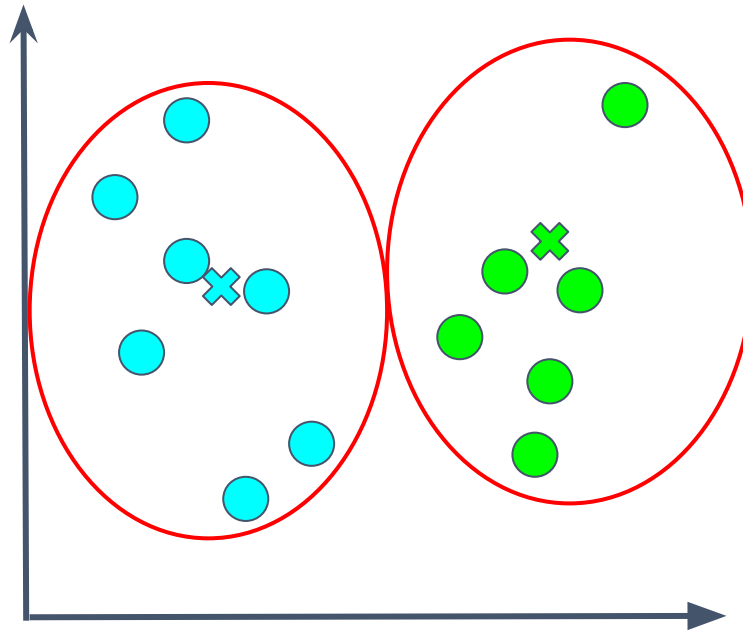
K-Means Clustering Algorithm steps

Algorithm 1 k -means algorithm

- 1: Specify the number k of clusters to assign.
 - 2: Randomly initialize k centroids.
 - 3: **repeat**
 - 4: **expectation:** Assign each point to its closest centroid.
 - 5: **maximization:** Compute the new centroid (mean) of each cluster.
 - 6: **until** The centroid positions do not change.
-

K-Means Clustering Visualized

Step 5: Repeat until centroid positions stop changing.



Stopping Criteria



If any of these are true, stop:

1. **Centroids of the newly formed clusters do not change**
 - Not learning any new pattern
2. **All data points stay in their same cluster**
 - No change to clusters
3. **Minimum amount of error reached** (*more on this in a min*)
 - Calculate Residual Sum of Squares (RSS, aka SSE) and check if small
4. **Maximum number of iterations are reached**
 - You hit a maximum number of iterations you want to run (usually < 50)

How do we **choose k**?

K-Means Clustering as an Optimization Problem

Minimize total **intra-cluster** variance, or **Within-Cluster Sum of Squares (WCSS)**

Where WCSS = sum of squares of the distances of each data point to its respective centroid for all clusters.

Optimization function for k-means clustering

$$J = WCSS = \sum_{j=1}^k \sum_{i=1}^n ||x_i^{(j)} - c_j||^2$$

Where

k = number of clusters

n = number of data points in cluster c_j

c_j = the centroid for cluster j

Assumptions:

- Each observation belongs to at least one of the k clusters
- Clusters do not overlap
- Variation within each cluster is minimized.

But, finding the global minimum of J is NP-hard, so we need another way.

Silhouette Method

- For all clusters:
 - For all data points in the cluster:
 - Calculate a score for the data point, indicating whether it is likely to be clustered well
 - Score Range: -1 to +1
 - +1 = Data point is clustered very well
 - - 1 = Data point is clustered poorly
 - 0 = Coin toss
 - Take the average across all data points in the cluster for the j^{th} cluster's quality
- Take the average cluster quality across all clusters for the overall Silhouette Score
- Choose k that produces the **highest overall Silhouette Score**

Pre-Processing and Post-Processing Tips

Pre-Processing

- Check for **Feature Scaling**
 - Clustering features with wildly different ranges/scales can lead to poor or unexpected results - be sure to normalize/standardize appropriately
- Check for **Outliers**
 - Recall from our discussion on Feature Scaling that Euclidean distance is very sensitive to outliers

Post-Processing

- Check for **small clusters**
 - Could indicate outliers or anomalies.
 - Can be helpful to merge small clusters into other clusters, depending on what these mean
- Check for **“loose” clusters**
 - Clusters with high errors can be split into multiple clusters
- Check for **clusters that are very close together**
 - Clusters that are close with low errors can often be merged

Pros and Cons of K-Means Clustering

+ Pros

- Easy to understand and implement
- Fast and scalable! → time complexity roughly linear
- Converges to local minimum of WCSS

– Cons

- Choosing k is key, but no way to know optimal k so we have to guess/approximate
- Sensitive to outliers
- Can be sensitive to initial choice of centroids (tend to go towards local minima nearby)
- Only as good as our definition of centroid/mean
 - For categorical values, the centroid is just the mode (most frequent)
- Not good at discovering clusters that are not spherical or ellipsoidal
 - Struggles with nested clusters or unusual shapes

“Testing” k-means

We can see how good our split is by:

- **The Elbow Method / WCSS:** Testing if adding more clusters (increasing k) actually helps or if it's just creating unnecessary complexity.
 - **Silhouette Score:** Testing how well-defined the boundaries are between your clusters.
 - **Stability Testing:** Running the algorithm 10 times with different starting points to see if it finds the same clusters every time. If it gives you totally different results each time, your "clusters" might just be random noise.
- 

“Testing” k-means

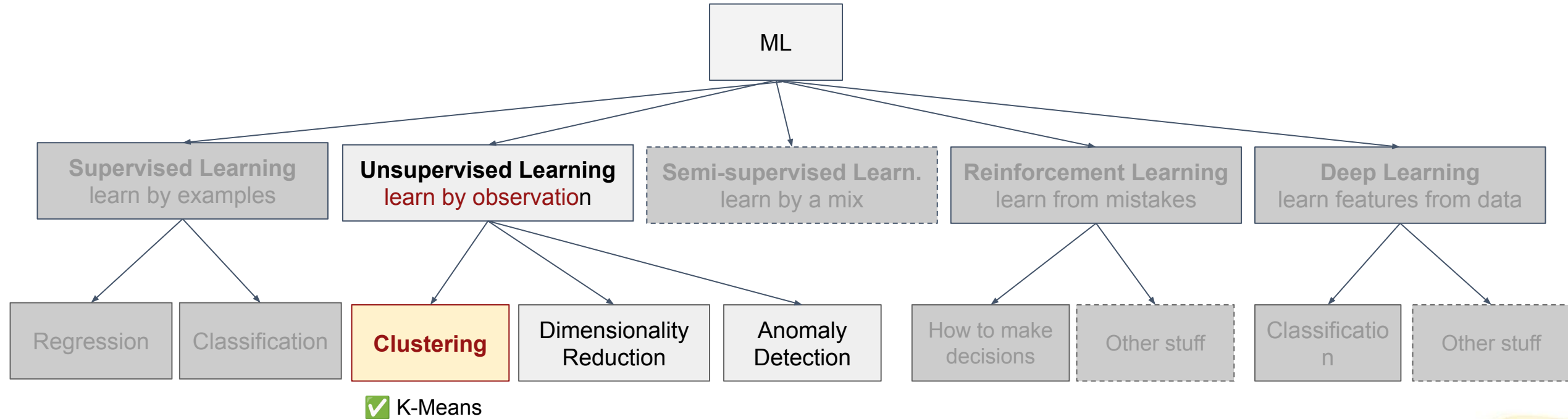
Sometimes, in advanced data science, we *do* use a separate set of data to see if the clusters are **robust**.

- **Step 1:** Run K-means on Dataset A and find 3 clusters.
- **Step 2:** Take Dataset B (which the model hasn't seen).
- **Step 3:** Assign points in Dataset B to the clusters found in Step 1.

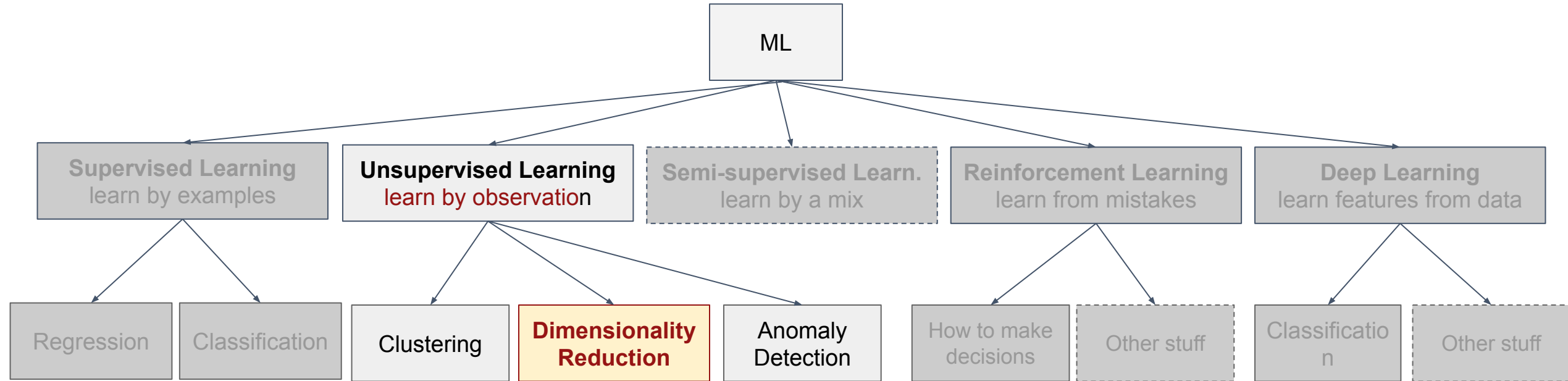
The Test: If Dataset B fits neatly into those same 3 shapes, the clusters are **generalized**. If Dataset B looks like a total mess when forced into those 3 shapes, your original clusters were probably just a fluke of the first dataset (overfitting).

Sanity check: if Dataset A has 20% of the points in cluster 1, 50% in cluster 2 and 30% in cluster 3, those proportions should be similar for Dataset B

Main categories of Machine Learning



Main categories of Machine Learning



Dimensionality Reduction



Dimensionality Reduction

The process of **reducing** the number of features (dimensions) in a dataset while **preserving its essential information**.



Recall: Dimensionality Reduction

Dimensionality: the **number of features** in the dataset.

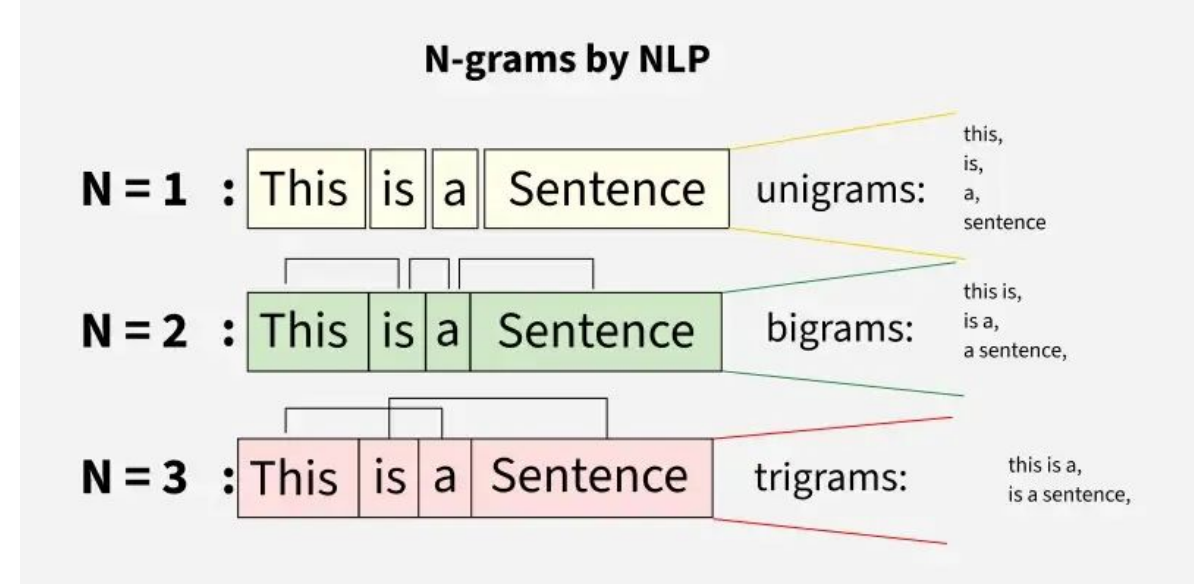
Dimensionality Reduction: **minimize this number.**

- Often we have features that are **correlated** (provide the same or related information) and this excessive or redundant information *adds complexity* and *increases noise*.
 - Examples:
 - "total purchase amount" and "number of items purchased"
 - "square footage" and "number of bedrooms"
 - Celsius and Fahrenheit temperatures
 - Represent the same physical quantity, just measured in different units.
- **Redundant and irrelevant features degrade performance** of many ML algorithms
 - Can lead to decreased efficiency and increased computational complexity.
 - Potential for overfitting
 - Curse of Dimensionality!

Many datasets are high-dimensional

Examples:

- **Document classification**
 - When features are words (unigrams), thousands per document
 - When features are bigrams, thousands per document
- **Magnetoencephalography (MEG) Brain Imaging**
 - Millions of features is common
- **Genomics**
 - Thousands of gene features for a single observation



Recall: The Good News

Most real-world applications have low-dimensional structure.

- Meaning there are often just a few features at most that contribute to the majority of the information we need

This means we can attempt to find **the fewest features that matter the most** for solving our problem.



Easy Wins

Sometimes, there are **easy things** we can do to **reduce dimensions**:

- Remove things we don't care about
 - ex. People's names
- Remove things that are obviously irrelevant
 - ex. A student's favorite color while we're doing GPA analysis



Harder Wins

In **Feature Engineering**, there are two ways for dimensionality reduction:

1. Feature Selection

- Select a **subset** of the most relevant features
- Could be done manually (e.g. remove one of two features that are correlated) or by an algorithm (e.g. Lasso regression)

2. Feature Extraction

- **Extract** (derive) features from the original dataset, preserving the original relationships and most significant information, and **map** the data onto a **new feature space**.

Harder Wins

In **Feature Engineering**, there are two ways for dimensionality reduction:

1. Feature Selection

- Select a **subset** of the most relevant features
- Could be done manually (e.g. remove one of two features that are correlated) or by an algorithm (e.g. Lasso regression)

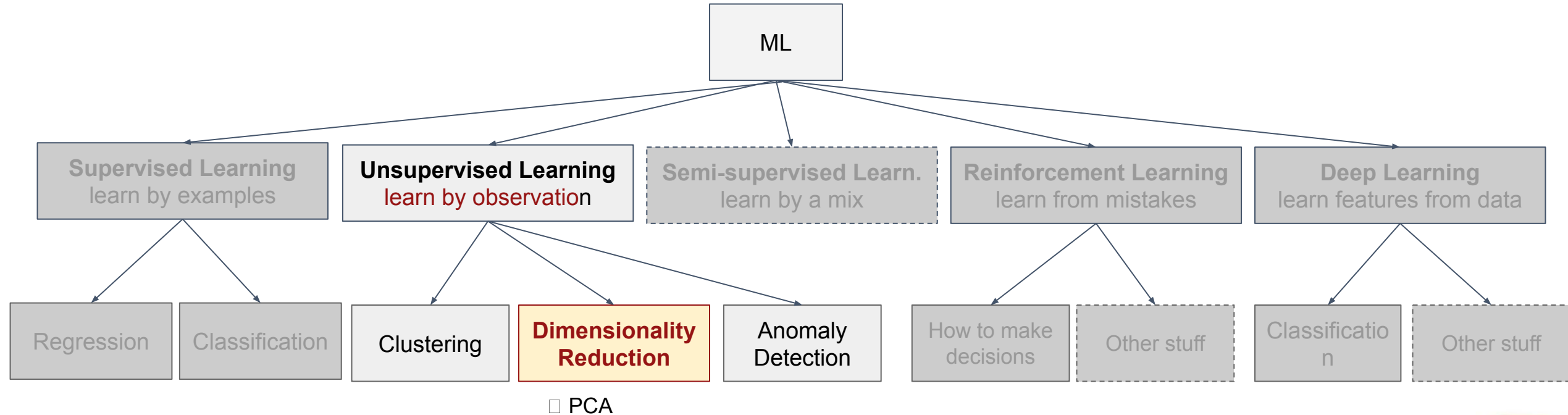
2. Feature Extraction

- **Extract** (derive) features from the original dataset, preserving the original relationships and most significant information, and **map** the data onto a **new feature space**.

We're going to expand **this**.

Principal Component Analysis (PCA)

Main categories of Machine Learning



Principal Component Analysis (PCA)

Goal: Reduce dimensionality while retaining essential information.

Approach: Combine features in a specific way, creating new features, called **Principal Components**, and remove the rest.

- To combine features, PCA uses a **linear transformation** to take an D -dimensional dataset and project it onto a lower d -dimensional subspace, while retaining the most important information.
- 

Motivating Example

Consider the following simple dataset with 3-dimensional data points:

	f_1	f_2	f_3
0	1	2	3
1	2	4	6
2	3	6	9
3	4	8	12
4	5	10	15

Suppose each value is stored using 1 byte.

→ 5 observations x 3 features = **15 bytes**

Motivating Example

If we look closely at this dataset, we can see that each data point is related. They are all the same data point, just scaled.

	f_1	f_2	f_3	
0	1	2	3	$= 1 * (1, 2, 3)$
1	2	4	6	$= 2 * (1, 2, 3)$
2	3	6	9	$= 3 * (1, 2, 3)$
3	4	8	12	$= 4 * (1, 2, 3)$
4	5	10	15	$= 5 * (1, 2, 3)$

Using this information, we could simply store the scaler along with the scale value for all the observations.

→ 1 scaler x 3 features + 5 values = 8 bytes.

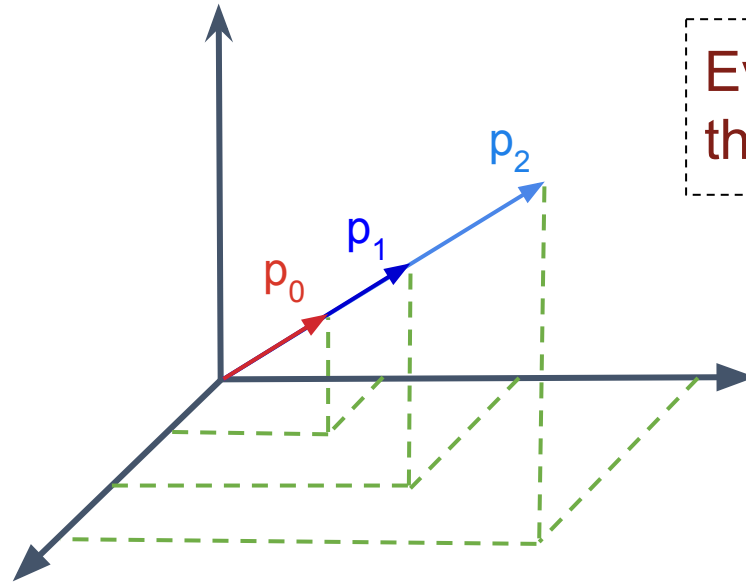
Almost 50% savings even for just a few examples!

Motivating Example

There is a geometric interpretation of this functionality.

Let's consider just 3 data points, for visual purposes:

p	f_1	f_2	f_3
0	1	2	3
1	2	4	6
2	3	6	9



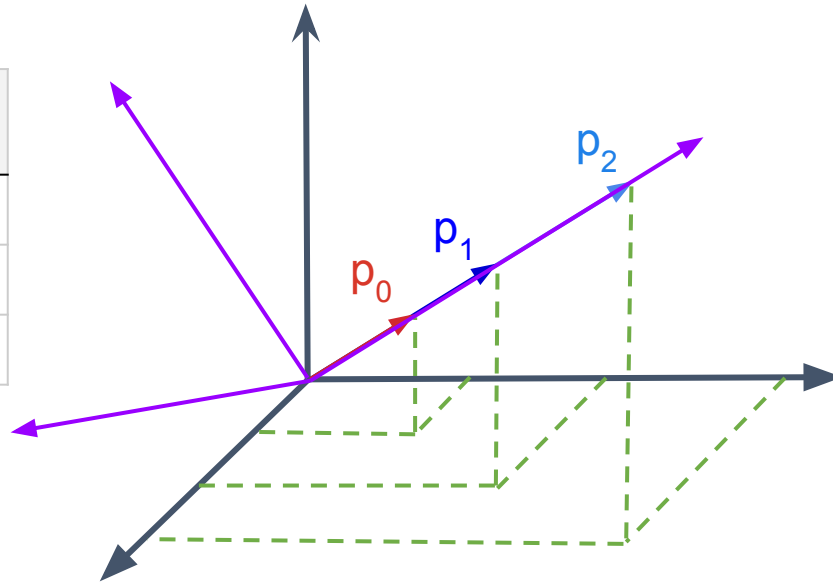
Even though each point is 3D, they all lie along a 1D line.

Motivating Example

There is a geometric interpretation of this functionality.

Let's consider just 3 points, for visual purposes:

p	f_1	f_2	f_3
0	1	2	3
1	2	4	6
2	3	6	9



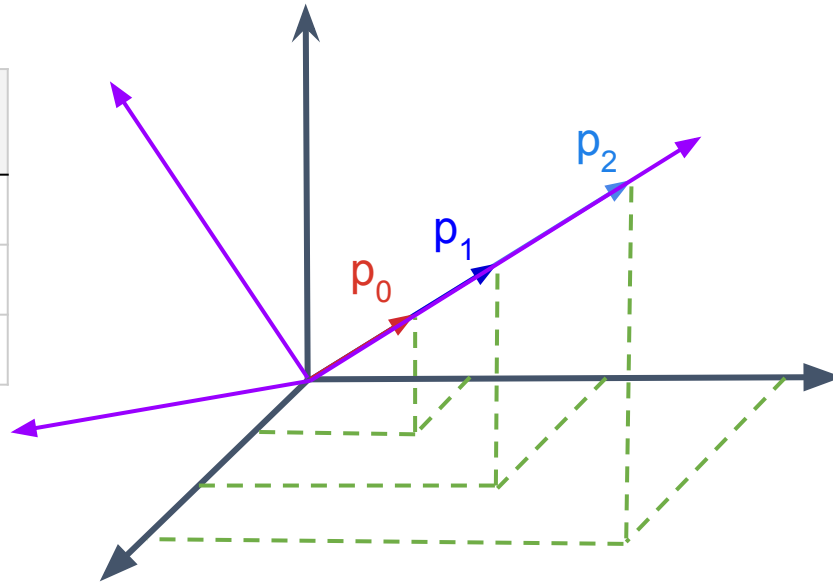
We could imagine a **new coordinate system** where this line is an axis.

Motivating Example

There is a geometric interpretation of this functionality.

Let's consider just 3 points, for visual purposes:

p	f_1	f_2	f_3
0	1	2	3
1	2	4	6
2	3	6	9



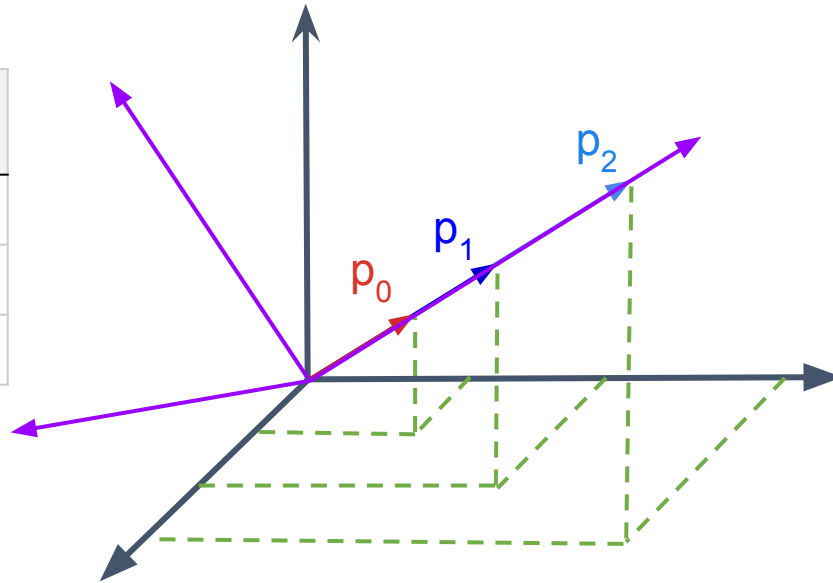
In this new coordinate system, each data point can be described by a single non-zero value along that axis (i.e. $p_0=1$, $p_1=2$, $p_2=3$).

Motivating Example

There is a geometric interpretation of this functionality.

Let's consider just 3 points, for visual purposes:

p	f_1	f_2	f_3
0	1	2	3
1	2	4	6
2	3	6	9



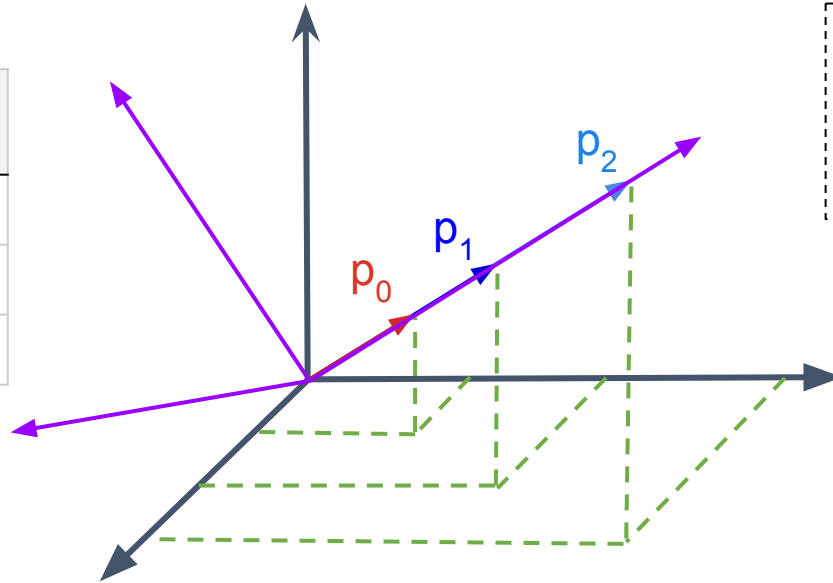
Now all we need to store is the transformation that created that axis (the direction) along with those values.

Motivating Example

There is a geometric interpretation of this functionality.

Let's consider just 3 points, for visual purposes:

p	f_1	f_2	f_3
0	1	2	3
1	2	4	6
2	3	6	9



Finding these axes is known as
Principal Component Analysis.

How do we find such projections?



PCA Big Idea

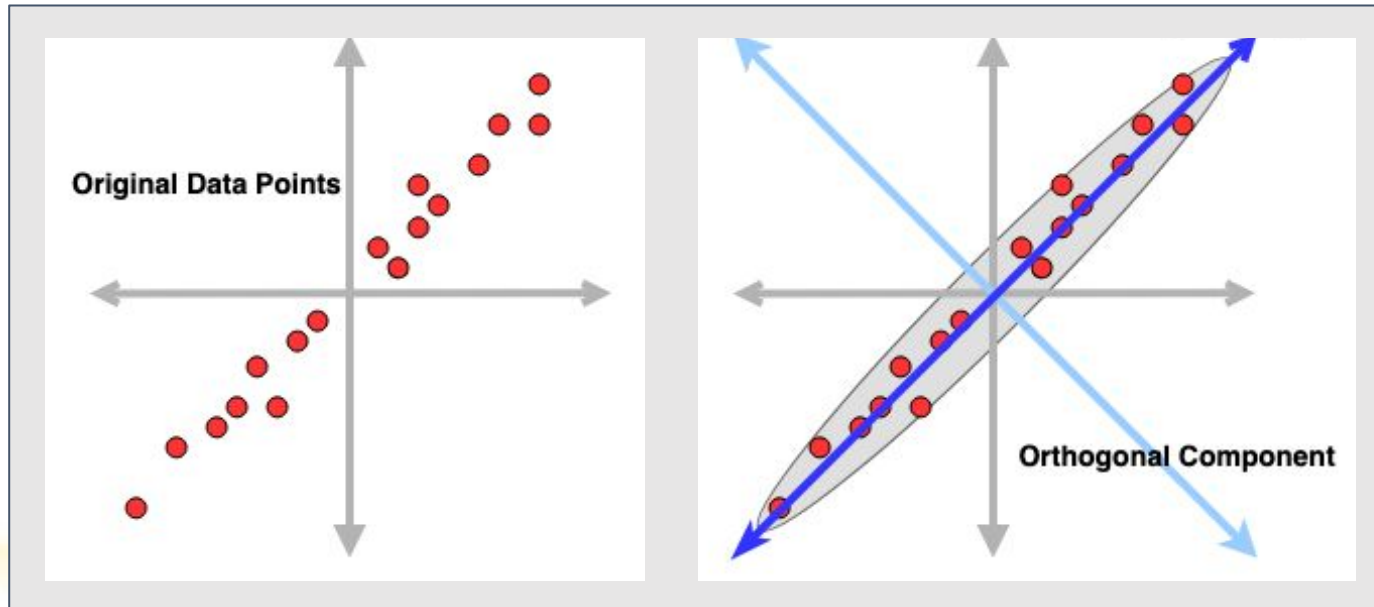
Transform the original features into a **new** set of **linearly uncorrelated** variables (**independent** and **orthogonal** to each other), called **Principal Components**, that capture the **maximum amount of variance** in the data.

- Look at the **correlation** between data points
 - High correlation indicates that there is redundancy in the data!
 - Highly correlated variables correspond to the direction of maximum variance
- Try to find **linear** combinations of the original features, ordered from highest to lowest variance (spread about the mean).
- Axes can be obtained using some **matrix math** (Eigenvalue or Singular Value Decomposition for a covariance matrix)
 - Features with the highest correlation (ex. eigenvectors with the largest eigenvalues) can be combined into a **Principal Component** (without redundancy or overlap with other components).

Principal Components (PCs)

Principal components are **new features** constructed as a **linear combination** of the **original features** that **captures the maximum variance**.

- Combinations are done in such a way that the new features (Principal Components) are **uncorrelated** and *most of the information within the initial variables is compressed into the first components*.



The PC is the **new axis** in the transformed space where the data variance is maximized.

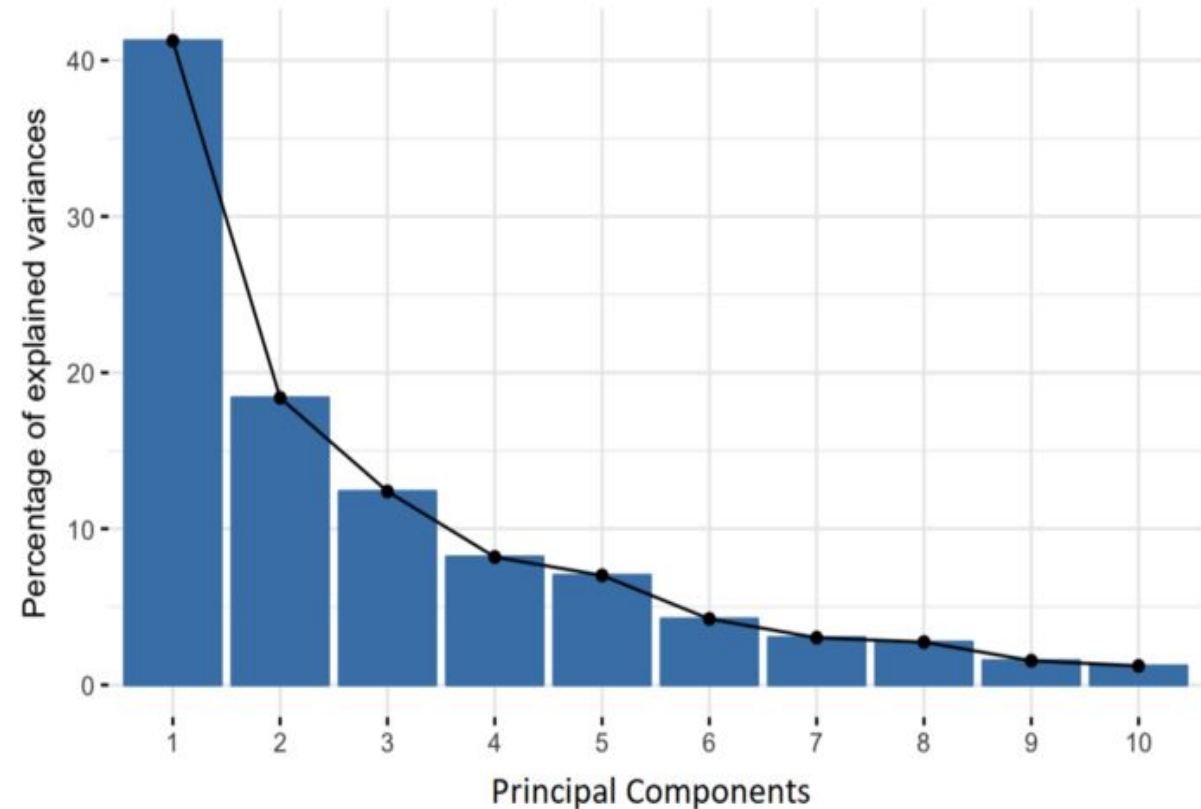
Each subsequent component is orthogonal to the previous ones, and captures the next highest variance.

Principal Components (PCs)

PCA tries to put the maximum possible information into the **first component**, then maximum remaining information into the second, and so on.

- The first PC will have the highest variance, the second PC will have the 2nd highest variance, etc.
- **Remember:** goal is to **combine related features but retain the most important info**
- Discard components with low variance, and consider the remaining components for the new variables.

Example for 10 PCs



Scree Plot: Percentage of Variance (Information) for each by PC

PCA Conceptual Algorithm

1. Find a line such that when the data is projected onto that line, it has maximum variance.
 2. Find a second line that is orthogonal to the first line, and has maximum variance of what's left after the first projection.
 3. Repeat until there are D orthogonal lines (original dimensionality).
 4. Pick the best d lines.
- 

PCA Steps

1. **Standardize** the data (if appropriate)
 2. Calculate the **covariance matrix**
 3. Compute the **eigenvectors** and **eigenvalues** of the covariance matrix
 4. Select the **Principal Components**
 5. Project the data onto the **new coordinate system** given by the Principal Components
- 

Step 1: Standardize the Data

Ensure the data is centered (mean of each variable is 0) and scaled (variance of each variable is 1).

$$Z = \frac{\bar{x} - \mu}{\sigma}$$

Why standardize?

1. If your features are in different units.
2. If the features' values have largely different scales (in which case a feature could dominate the variance).

Step 2: Calculate the Covariance Matrix

Let's first set up what we're aiming to do.

Remember: our data is a matrix.

If we have n observations and D features, the data matrix is $n \times D$.

- Each column is a set of measurements across n observations for a single feature
- Each row is a single observation across all D features.

Recall that the sample **mean** (average value) of a single feature is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

And the sample **variance** (how spread out the data is) across the different measurements for that feature is

$$var(x) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Notice this is similar to Mean Squared Error (MSE)!

Covariance

Suppose we look at 2 of the D feature columns (vectors).

- Let's call one feature vector x , and the other y .

We can measure the relationship between the two (to what extent they vary together) using sample **Covariance**:

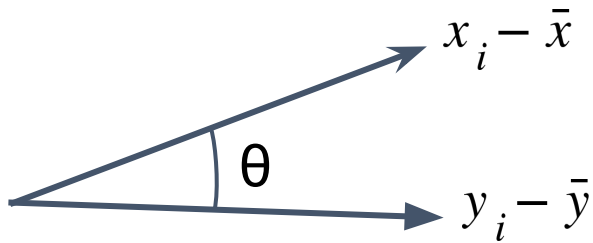
$$\text{cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- A **positive** covariance shows that both features vary in the **same** direction.
- A **negative** covariance shows that the features vary in the **opposite** direction.
- **Zero** covariance shows they are not varying together.
- Values can be anything on $[-\infty, \infty]$

Correlation

$$\text{cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Notice that **covariance** actually contains a dot product of 2 vectors.



The angle between these vectors is given by **$r = \cos(\theta)$** where r is the Pearson **Correlation Coefficient**

Recall that $a \cdot b = \|a\| \|b\| \cos(\theta)$

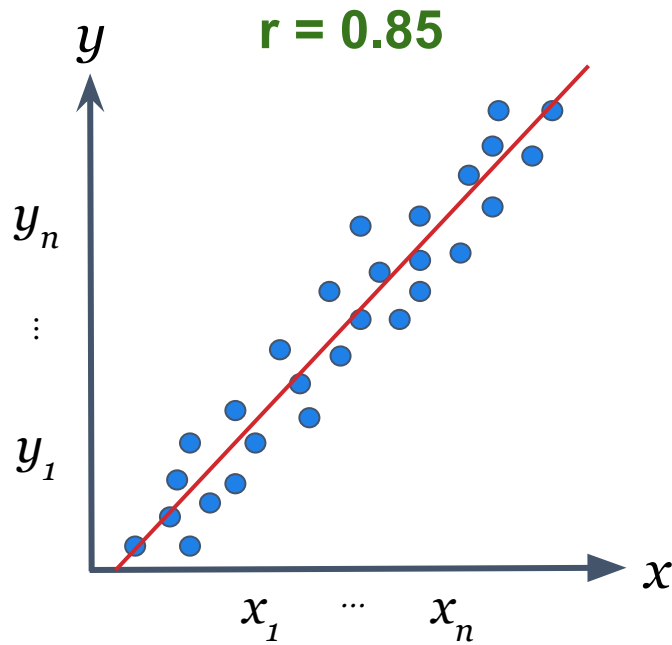
Covariance tells us the **direction** of the relationship.

Correlation gives us the **strength** of the relationship in addition to the **direction**.

$$r = \cos(\theta) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}} = \frac{\text{cov}(x, y)}{\sqrt{\text{var}(x) \text{var}(y)}}$$

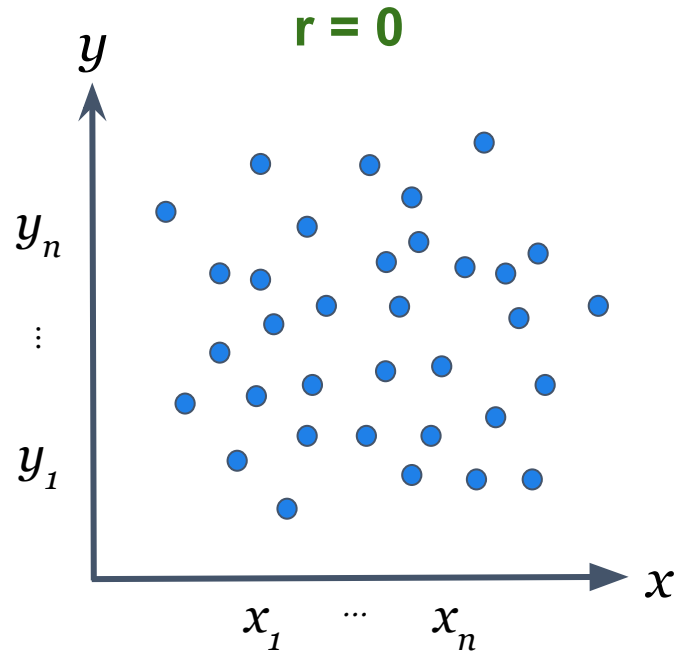
Correlation can be thought of as covariance **normalized** onto $[-1, 1]$.

Intuition



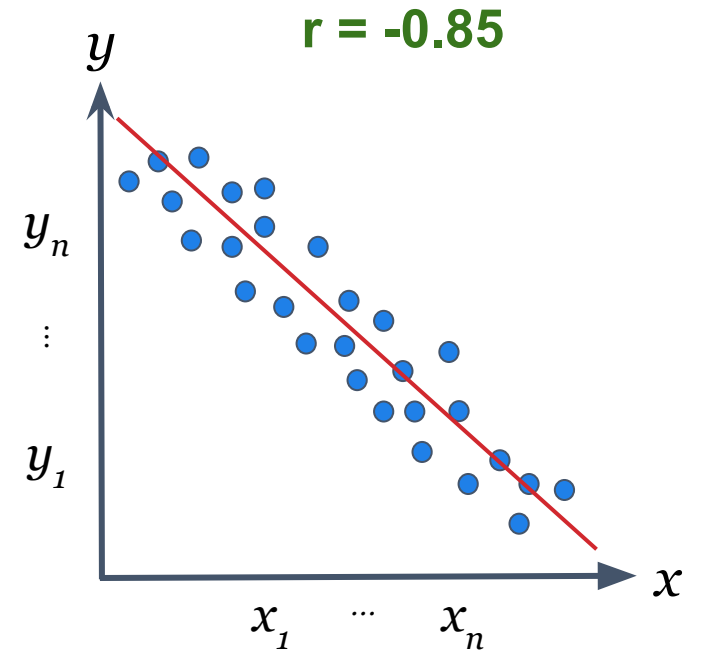
Positive Covariance
Positive Correlation

Linear relationship:
as x increases, y increases



Zero Covariance
Zero Correlation

Not a linear relationship



Negative Covariance
Negative Correlation

Linear relationship:
as x increases y decreases

If $r=1$, the angle between x and y is 0, and thus they are linearly dependent and redundant!

Covariance Matrix

Because covariance (and correlation) compare 2 features at a time, if we want to look at all the feature combinations, we'll need to calculate the **pairwise covariances**.

For our simple example of just 2 features, x and y , this creates a 2x2 matrix:

$$\begin{bmatrix} \text{var}(x) & \text{cov}(x, y) \\ \text{cov}(x, y) & \text{var}(y) \end{bmatrix}$$

The diagonal is variance because the covariance of a variable and itself is just the variance of that variable.

This matrix is actually also created by linear regression when minimizing MSE.

Covariance Matrix

More generally, the covariance matrix is a square, symmetric $D \times D$ matrix, often denoted Σ , that provides information about the variance of all D features along different axes.

- Each element represents the **covariance** between two features → **the direction** of the linear relationship between them.
- When the data is standardized (means are zero), the covariance matrix equals the correlation matrix (matrix where diagonals are 1 and off-diagonal elements are r).

$$\begin{bmatrix} \text{var}(x_1) & \dots & \text{cov}(x_D, x_1) \\ \vdots & \ddots & \vdots \\ \text{cov}(x_D, x_1) & \dots & \text{var}(x_D) \end{bmatrix}$$

Diagonal Elements (Variance): Represent the variance (spread) of the data along each axis (dimension).

Off-Diagonal Elements (Covariance): Represent the covariance between different pairs of dimensions, indicating how much the two dimensions **vary** (or change) **together**.

Covariance Matrix Calculation

To calculate the Covariance Matrix Σ , we simply plug in the various values.

We can also use the following equation by letting X be our $n \times D$ original dataset, and $\bar{x}$ be the vector of D feature means across n observations (a $1 \times D$ vector of mean values).

$$\Sigma = \frac{1}{n-1} (X - \bar{x})^T (X - \bar{x}) = \begin{bmatrix} \text{var}(x_1) & \dots & \text{cov}(x_D, x_1) \\ \vdots & \ddots & \vdots \\ \text{cov}(x_D, x_1) & \dots & \text{var}(x_D) \end{bmatrix}$$

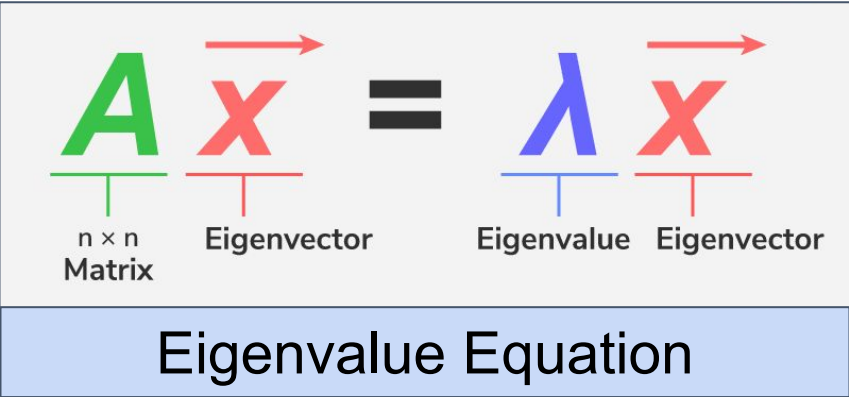
Why the covariance matrix is used in PCA

- **Variance capture:** PCA aims to find the directions in which the variance of the data is maximized. The covariance matrix helps identify these directions.
 - **Orthogonality:** PCA requires the Principal Components to be orthogonal to each other. The covariance matrix helps identify these orthogonal directions.
 - **Eigenvalue decomposition:** Principal Components can be obtained by performing simple matrix math (eigenvalue decomposition) on the covariance matrix.
- 

Linear Algebra Terminology Refresher

In linear algebra, an **eigenvector** is a vector whose **direction is unchanged** by a given linear transformation.

- In other words, when an eigenvector is multiplied by a matrix, it only gets scaled by a constant factor.
- This constant factor is called an **eigenvalue**, typically denoted λ .



The diagram illustrates the Eigenvalue Equation. It shows a green matrix A (labeled $n \times n$ Matrix) multiplied by a red vector $\vec{x}$ (labeled Eigenvector), which is equal to a blue scalar λ (labeled Eigenvalue) multiplied by the same red vector $\vec{x}$ (labeled Eigenvector). The entire equation is enclosed in a light gray box, and the text "Eigenvalue Equation" is written in a blue box below it.

$$\underbrace{A}_{n \times n \text{ Matrix}} \underbrace{\vec{x}}_{\text{Eigenvector}} = \underbrace{\lambda}_{\text{Eigenvalue}} \underbrace{\vec{x}}_{\text{Eigenvector}}$$

Eigenvalue Equation

Step 3: Compute the Eigenvectors and Eigenvalues of the Covariance Matrix

The next step is to decompose the covariance matrix into **eigenvalues** and **eigenvectors**. This process is called **Eigenvalue decomposition**.

Why do this?

- The Covariance Matrix defines both the **spread** (variance) and the **orientation** (covariance) of our data.
- By finding the eigenvector with the largest eigenvalue in the covariance matrix, we identify **the direction of the maximum variance**.
- This direction defines a **new axis** onto which we can project the data.
- Remember that **Principal Components** are these new axes (new combinations of linearly related features)

Eigenvectors and Eigenvalues in PCA

Eigenvectors: Indicate the **directions of maximum variance** (most information) in the data (**Principal Components**).

Eigenvalues: tells us how much variance (**magnitude** of variance) is captured along the direction of its corresponding eigenvector/Principal Component.

Properties:

- Every eigenvector has a corresponding eigenvalue.
- The number of eigenvector-eigenvalue pairs equals the number of dimensions (features) in the data.
 - Ex. a 3-D dataset has 3 eigenvectors and 3 corresponding eigenvalues.

Step 3.1: Eigenvalue Decomposition (EVD)

EVD decomposes a square matrix (like the $D \times D$ covariance matrix) into its eigenvectors and eigenvalues, by solving for Σ in the eigenvalue equation:

$$\Sigma \mathbf{v} = \lambda \mathbf{v}$$

Σ is the covariance matrix.

$\mathbf{v}$ is an eigenvector of Σ .

λ is the corresponding eigenvalue of Σ .



$$\Sigma \mathbf{v} - \lambda \mathbf{v} = (\Sigma - \lambda \mathbf{I}) \mathbf{v} = \mathbf{0}$$

$\mathbf{I}$ is the identity matrix

We can use the determinant of the above equation to solve first for the eigenvalues, then plug in to solve for the eigenvectors.

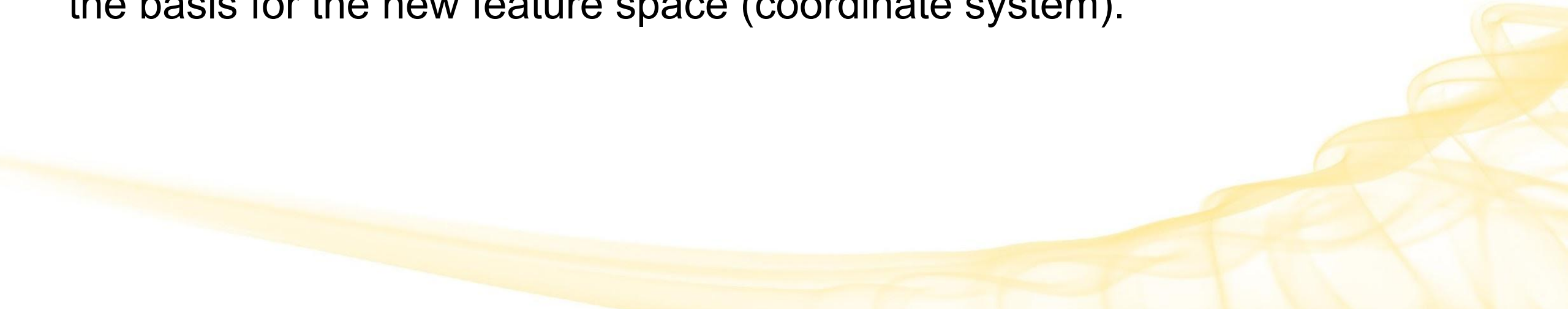
We won't go over the math in detail here.

Step 4: Select the Principal Components

Once we have the eigenvectors and eigenvalues:

- **Rank the eigenvectors:** Order eigenvectors by eigenvalues, **highest to lowest**.
 - i.e. rank such that $\lambda_1 \geq \lambda_2 \geq \dots \lambda_D$
- **Selection:** Choose top ***d*** eigenvectors (Principal Components).

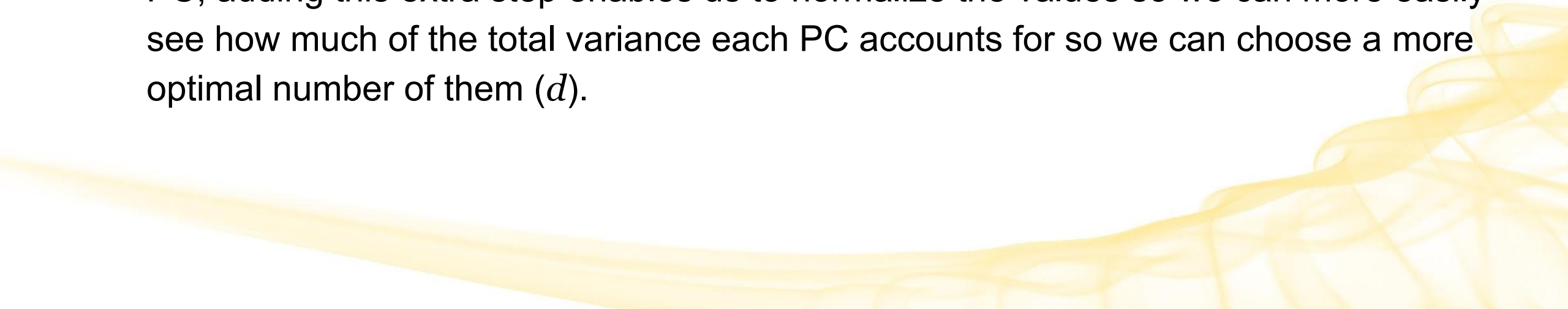
These selected eigenvectors capture the most variance in the data and form the basis for the new feature space (coordinate system).



Step 4.1: Check Variance Contribution of PCs

After obtaining the Principal Components, it's important to assess each one's contribution to the overall variance.

Why?

- PCA aims to capture the most important information using the fewest dimensions.
 - Some PCs are more useful than others, capturing variance rather than noise.
 - While the eigenvalues directly represent the amount of variance explained by each PC, adding this extra step enables us to normalize the values so we can more easily see how much of the total variance each PC accounts for so we can choose a more optimal number of them (d).
- 

Variance Contribution of Principal Components

Once Principal Components are selected, we **calculate the percentage of variance accounted for by each**:

- Divide the eigenvalue of each PC by the sum of all eigenvalues

$$\lambda_k / \sum_i \lambda_i = \text{proportion of variance captured by } k^{\text{th}} \text{ PC}$$

- Multiply this by 100 to get percentage.
- Sort the eigenvectors by eigenvalues, highest to lowest $\lambda_1 \geq \lambda_2 \geq \dots \lambda_D$

Step 4.2: Dimensionality Reduction

To decide how many PCs to keep, we look at each one's contribution.

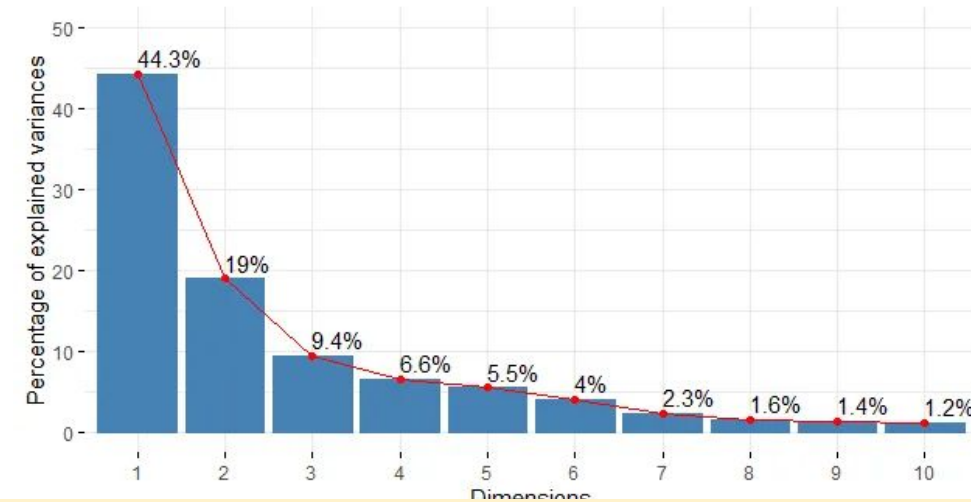
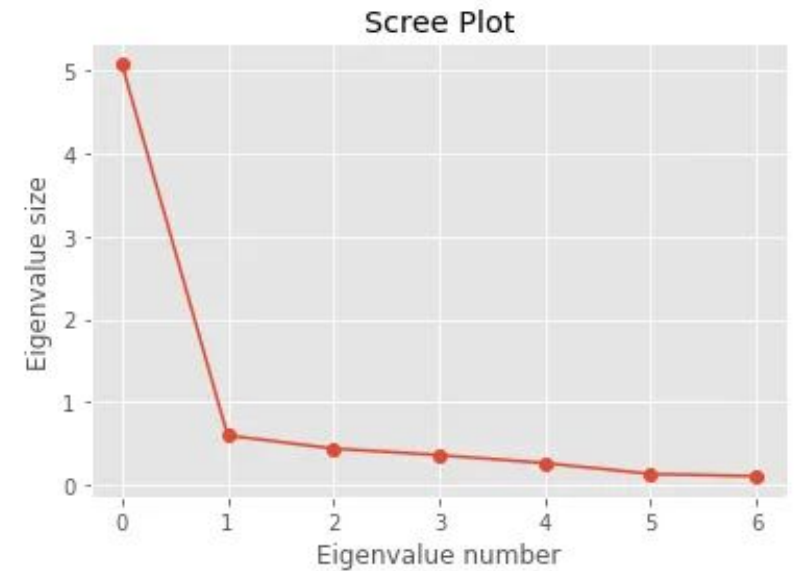
- We **keep the PCs that collectively capture most of the variance** and discard those with minimal contribution.

Ignore the PCs with less significance (those that appear at the bottom of the PC list) due to their low eigenvalues.

- This will reduce the original dataset of $(x_1, x_2, \dots, x_D)$ variables to a smaller dataset with $(PC_1, PC_2, \dots, PC_d)$, where $d < D$.

How to decide top d eigenvectors

1. **Elbow Method:** Create a Scree plot
2. **Calculate the percentage of explained variance and analyze the contribution percentages:**
 - **Common approach:** keep PCs that together explain a specific percentage of the total variance (ex. 80% or 90%).
 - Choose threshold based on needs: higher for comprehensive information, lower for simpler interpretation.



Each bar shows the explained variance percentage of individual components. The red line is a type of scree plot. By analyzing this plot, we decide to keep only the first two or three components because they capture almost all the variance in the dataset.

Step 4.3: Create a Projection Matrix

Now that we have selected our final set of d PCs (eigenvectors), we can easily create a projection matrix W whose columns are these PCs we want to use.

- $W = [PC_1, PC_2, \dots, PC_d]$

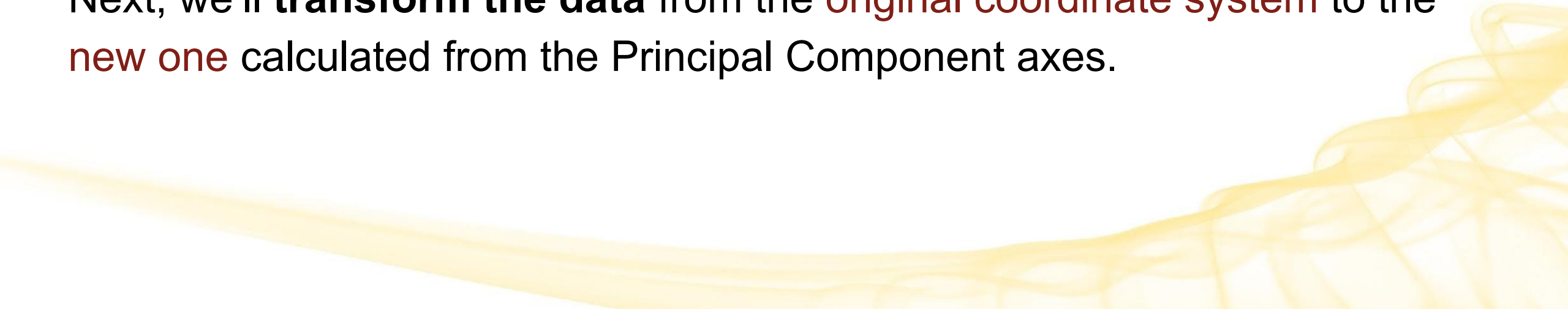
We'll use this matrix to perform our feature transformation to take our dataset from D dimensions to d dimensions.

Transformation

So far, apart from possible standardization, we haven't made any changes to the original data.

- We just selected the Principal Components and formed a projection matrix with them.
- However, the initial data remains on the original axes.

Next, we'll **transform the data** from the **original coordinate system** to the **new one** calculated from the Principal Component axes.



Step 5: Project the data

Now we **project the data onto the new coordinate system** given by the Principal Components, using the projection matrix W that we created in the previous step.

We simply multiple the (possibly standardized) dataset X by the projection matrix to create our new, lower-dimensional, transformed dataset T using matrix multiplication:

$$T = X_{\text{standardized}} * W$$

This transformation re-expresses the data in terms of its Principal Components, which are the new axes that capture the most variance in the data.

- The 1st column of T is the projection of the data onto the 1st PC
- The 2nd column of T is the projection onto the 2nd PC, and so on.

Recap of PCA Steps

1. **Standardize** the D -dimensional data (if appropriate)
 2. Calculate the $D \times D$ **covariance matrix**
 3. Compute the **eigenvectors** and **eigenvalues** of the covariance matrix
 4. Select the top d **Principal Components**
 5. Project the data onto the d -dimensional **new coordinate system** given by the Principal Components
- 

PCA Python Example

.ipynb



Benefits of PCA

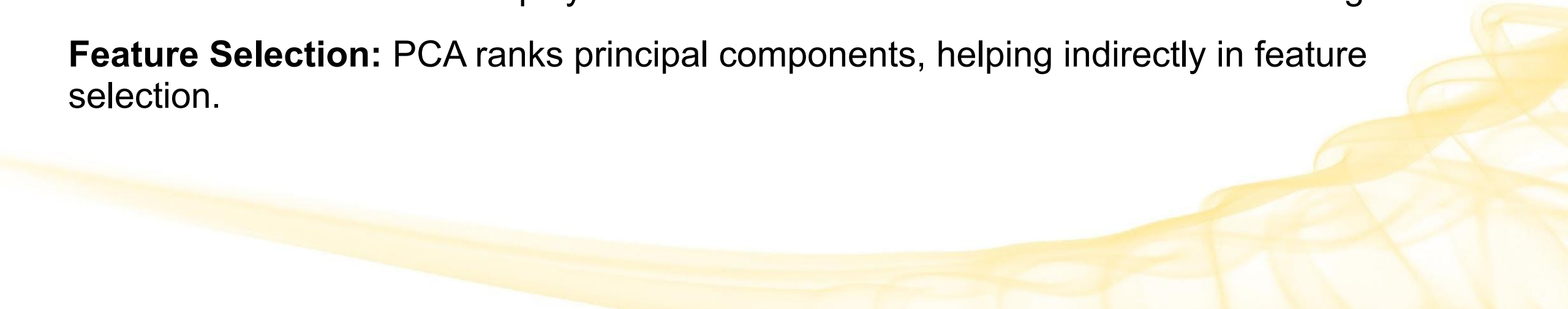
Dimension Reduction: PCA reduces the number of features while retaining essential information, making datasets more manageable.

Noise Reduction: eliminates noise in data, improving feature quality and model performance.

Feature Transformation: transforms original features into orthogonal components, potentially revealing hidden patterns.

Visualization: PCA can simplify data into 2 or 3D for easier visual understanding.

Feature Selection: PCA ranks principal components, helping indirectly in feature selection.



Limitations of PCA

Linearity: PCA is a linear transformation so it works better if your features are related linearly.

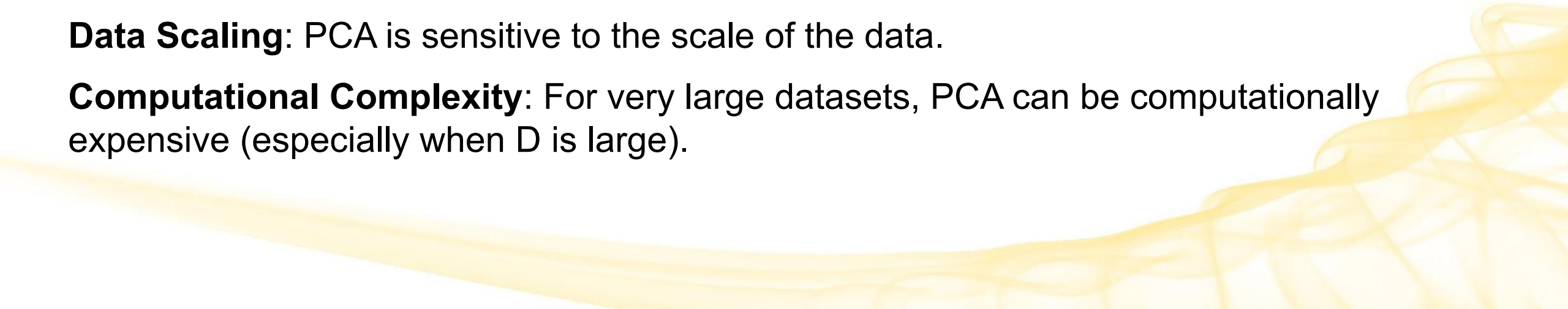
Interpretability: Principal components can be hard to understand in terms of original features.

Loss of Information: PCA might lose important information (could lead to oversimplification or distortion of the data), depending on the number of PCs selected.

Orthogonality: PCA assumes that Principal Components are orthogonal.

Data Scaling: PCA is sensitive to the scale of the data.

Computational Complexity: For very large datasets, PCA can be computationally expensive (especially when D is large).



PCA is common

In the world of data science and machine learning, PCA (Principal Component Analysis) is the most common dimensionality reduction technique.

You will find it used in almost every industry that deals with high-dimensional data.



Top 3 use cases

- **Visualization:** Humans can't visualize 50 dimensions. PCA is used to squash those 50 variables down to 2 or 3 Principal Components so we can plot the data on a graph and see if there are any obvious clusters or patterns.
- **Noise Reduction:** By keeping only the top components (the ones with the most variance), PCA effectively "filters out" the small, random fluctuations (noise) in a dataset, leaving behind the core signal. Helps with overfitting!
- **Speeding up Models:** If you are training a complex model (like a Random Forest) on a dataset with 500 features, it might take hours. If PCA can represent 95% of that information in just 20 features, your model will run significantly faster without losing much accuracy.

To PCA or not to PCA?

What you GAIN	What you LOSE
Speed: Faster model training.	Information: Some data is discarded.
Visualization: Can see 10D in 2D.	Meaning: Features become abstract.
Simplicity: Fewer variables to manage.	Nuance: Tiny but important signals might vanish.

Main categories of Machine Learning

